ALGORITHMIQUE EN BASH · JOUR 1 SUR 5 · LUNDI 19 OCTOBRE

Avant de commencer

Un algorithme, c'est une recette : une suite d'étapes précises qui transforme des données d'entrée en résultat. On va en écrire beaucoup cette semaine, en bash. Le langage n'a rien d'idéal pour ça (il est lent, ses calculs ne connaissent que les entiers), mais vous l'avez pratiqué pendant la formation Linux et il oblige à tout écrire soi-même. Vous savez déjà taper `cut -d: -f7 /etc/passwd | sort | uniq -c`. Cette semaine, on regarde ce qu'il y a à l'intérieur de `sort` et de `uniq` : si on veut trier, on écrit le tri.

Le principe de chaque séance : une notion, une démo en partage d'écran, puis des exercices à faire seuls. Les exercices sont notés de 1 à 3 selon leur difficulté. Les niveaux 3 sont là pour ceux qui ont fini, personne n'est obligé de tout boucler.

Votre environnement

Rien à installer. Vous travaillez dans votre VM Ubuntu Server de la formation Linux, ou directement sur votre machine si elle tourne déjà sous Linux. Éditeur au choix : nano ou vim.

Tout le support suppose bash 5 (Ubuntu 24.04 et Arch le fournissent). Vérifiez quand même :

```
bash --version
```

Rappel : lancer un script

bonjour.sh

```
#!/usr/bin/env bash
# La première ligne (le « shebang ») indique quel programme doit lire le fichier.
echo "Bonjour depuis mon premier script"
```

Sortie

```
Bonjour depuis mon premier script
```

On le rend exécutable une fois, puis on le lance :

```
chmod +x bonjour.sh
./bonjour.sh
```

Quand un script ne fait pas ce que vous pensez, lancez-le avec `bash -x bonjour.sh`. Bash affiche alors chaque ligne exécutée, avec les valeurs des variables déjà remplacées. C'est le meilleur outil de débogage de la semaine, de loin.

Lundi · Rappels, conditions, boucles, tableaux

Variables, arguments, tests, boucles : vous avez vu tout ça au jour 7 de la formation Linux. On le reprend vite, avec une syntaxe de test différente de celle que vous connaissez (section 1). L'après-midi, on attaque les tableaux, qui serviront toute la semaine.

1. `[ ]` ou `(( ))` ?

En formation Linux, vous avez sûrement écrit vos tests avec `[ ]` et des options comme `-eq`, `-lt` ou `-f`. Ça marche, et pour tester un fichier, `[ -f fichier ]` reste la bonne syntaxe. Pour comparer des nombres et calculer, ce support utilise `(( ))` : on écrit `<` et `==` comme en maths, et les variables n'ont même pas besoin du `$`. Pour les chaînes, on prend `[[ ]]`, la version améliorée de `[ ]`.

Avec `[ ]`	Ce qu'on écrit cette semaine
`[ $a -eq $b ]`, `[ $a -ne $b ]`	`(( a == b ))`, `(( a != b ))`
`[ $a -lt $b ]`, `-le`, `-gt`, `-ge`	`(( a < b ))`, `<=`, `>`, `>=`
`[ $a -gt 0 ] && [ $a -lt 10 ]`	`(( a > 0 && a < 10 ))`
`[ "$s" = "oui" ]`	`[[ $s == "oui" ]]`
`[ -z "$s" ]`, `[ -n "$s" ]`	`[[ -z $s ]]`, `[[ -n $s ]]`
`[ -f fichier ]`, `[ -d dossier ]`	pareil, ou `[[ -f fichier ]]`

Pourquoi changer ? Parce que `[ ]` réserve des surprises dès qu'on y met `<` ou `>`. Ce script a l'air de comparer 5 et 9 :

lundi_crochets.sh

```bash
#!/usr/bin/env bash
a=5
b=9
if [ $a > $b ]; then
  echo "[ ] affirme que $a > $b"
fi
if [[ -f $b ]]; then
  echo "et un fichier nommé « $b » vient d'apparaître"
  rm "$b"
fi
if (( a > b )); then
  echo "(( )) affirme que $a > $b"
else
  echo "(( )) répond correctement : $a n'est pas plus grand que $b"
fi
```

Sortie

```
[ ] affirme que 5 > 9
et un fichier nommé « 9 » vient d'apparaître
(( )) répond correctement : 5 n'est pas plus grand que 9
```

Dans `[ ]`, le `>` est une redirection. Bash a créé un fichier vide nommé `9`, puis il a testé `[ 5 ]`, qui est toujours vrai puisque la chaîne n'est pas vide.

2. Variables et entrées

Une variable se crée avec `=`, **sans espace autour**. On lit sa valeur avec `$`. Pour récupérer ce que tape l'utilisateur, `read`. Pour récupérer ce qu'on a passé au script sur la ligne de commande, `$1`, `$2`, etc.

lundi_variables.sh

```bash
#!/usr/bin/env bash
read -p "Votre prénom : " prenom
read -p "Votre année de naissance : " annee
age=$(( 2026 - annee ))
echo "Bonjour $prenom, vous avez $age ans cette année."
```

Sortie avec « Robin » puis « 1998 » tapés au clavier

```
$ ./lundi_variables.sh
Bonjour Robin, vous avez 28 ans cette année.
```

lundi_arguments.sh

```bash
#!/usr/bin/env bash
ville=$1
departement=$2
echo "Vous habitez $ville, dans le $departement."
```

Exemple d'exécution

```
$ ./lundi_arguments.sh Angers 49
Vous habitez Angers, dans le 49.
```

Trois erreurs que tout le monde fait le premier jour :

```bash
nom = "Robin"      # bash cherche une commande appelée « nom » : erreur
nom="Robin"        # correct
echo '$nom'        # affiche $nom tel quel : les apostrophes bloquent tout
echo "$nom"        # affiche Robin
```

3. Calculer

Les calculs se font entre `$(( ))`. Bash ne connaît que les entiers : `17 / 5` donne 3, point.

lundi_calculs.sh

```bash
#!/usr/bin/env bash
a=17
b=5
echo "Somme     : $(( a + b ))"
echo "Produit   : $(( a * b ))"
echo "Quotient  : $(( a / b ))"    # division entière
echo "Reste     : $(( a % b ))"    # le modulo, très utile
echo "Puissance : $(( a ** 2 ))"
(( a++ ))                          # (( )) sans $ : on calcule sans afficher
echo "a vaut maintenant $a"
```

Sortie

```
Somme     : 22
Produit   : 85
Quotient  : 3
Reste     : 2
Puissance : 289
a vaut maintenant 18
```

Le modulo `%` revient tout le temps en algorithmique. `n % 2` vaut 0 si `n` est pair. `n % 10` donne le dernier chiffre de `n`.

4. Conditions

Même structure `if / elif / else / fi` qu'en formation Linux. Seuls les tests changent, comme vu en section 1.

lundi_mention.sh

```bash
#!/usr/bin/env bash
note=$1
if (( note < 0 || note > 20 )); then
  echo "Note invalide"
elif (( note >= 16 )); then
  echo "Très bien"
elif (( note >= 12 )); then
  echo "Bien"
elif (( note >= 10 )); then
  echo "Passable"
else
  echo "Insuffisant"
fi
```

Exemple d'exécution

```
$ ./lundi_mention.sh 14
Bien
$ ./lundi_mention.sh 21
Note invalide
```

Un ajout utile de `[[ ]]` : `[[ $s == *abc* ]]` teste si `s` contient « abc ».

Quand on compare une variable à une liste de valeurs possibles, `case` est plus lisible qu'une série de `elif` :

lundi_case.sh

```bash
#!/usr/bin/env bash
read -p "Continuer ? (o/n) " reponse
case $reponse in
  o|O|oui) echo "On continue." ;;
  n|N|non) echo "On s'arrête." ;;
  *)       echo "Réponse non comprise : $reponse" ;;
esac
```

Sortie avec « O » tapé au clavier

```
$ ./lundi_case.sh
On continue.
```

5. Boucles

Trois formes à connaître. `for` sur une plage, `for` « à la C » quand la borne est dans une variable, et `while` quand on ne sait pas à l'avance combien de tours il faudra.

lundi_boucles.sh

```bash
#!/usr/bin/env bash
for i in {1..5}; do
  echo -n "$i "
done
echo

n=4
for (( i = n; i >= 1; i-- )); do
  echo -n "$i "
done
echo

# On ajoute 1, 2, 3... jusqu'à dépasser 20
total=0
k=1
while (( total < 20 )); do
  (( total += k ))
  (( k++ ))
done
echo "total=$total après $(( k - 1 )) ajouts"
```

Sortie

```
1 2 3 4 5
4 3 2 1
total=21 après 6 ajouts
```

Le piège classique. Les accolades sont traitées avant les variables, donc ceci ne marche pas :

lundi_piege.sh

```bash
#!/usr/bin/env bash
n=4
for i in {1..$n}; do
  echo "$i"
done
```

Sortie

```
{1..4}
```

`continue` passe directement au tour suivant, `break` sort de la boucle :

lundi_break.sh

```bash
#!/usr/bin/env bash
# De 1 à 20, on saute les multiples de 3 et on s'arrête après 14
for (( i = 1; i <= 20; i++ )); do
  if (( i % 3 == 0 )); then continue; fi
  if (( i > 14 )); then break; fi
  echo -n "$i "
done
echo
```

Sortie

```
1 2 4 5 7 8 10 11 13 14
```

Pour les affichages alignés, `printf` fait mieux que `echo` : `printf '%3d\n' 7` affiche « 7 » sur trois caractères.

6. Tableaux

Nouveau par rapport à la formation Linux. Un tableau range plusieurs valeurs sous un seul nom, chacune repérée par son indice.

mardi_tableaux.sh

```bash
#!/usr/bin/env bash
notes=(12 8 15 10 17)
echo "Première note : ${notes[0]}"     # les indices commencent à 0
echo "Toutes : ${notes[*]}"
echo "Nombre : ${#notes[@]}"

notes+=(9)                             # ajout à la fin
notes[1]=11                            # modification

somme=0
for n in "${notes[@]}"; do             # parcours par valeur
  (( somme += n ))
done
echo "Somme : $somme"

for i in "${!notes[@]}"; do            # parcours par indice
  echo "notes[$i] = ${notes[i]}"
done
```

Sortie

```
Première note : 12
Toutes : 12 8 15 10 17
Nombre : 5
Somme : 74
notes[0] = 12
notes[1] = 11
notes[2] = 15
notes[3] = 10
notes[4] = 17
notes[5] = 9
```

Premier vrai algorithme : chercher le maximum. On suppose que le premier élément est le plus grand, puis on le remplace dès qu'on trouve mieux.

mardi_max.sh

```bash
#!/usr/bin/env bash
valeurs=(14 3 27 8 27 1)
max=${valeurs[0]}
pos=0
for i in "${!valeurs[@]}"; do
  if (( valeurs[i] > max )); then
    max=${valeurs[i]}
    pos=$i
  fi
done
echo "Maximum $max, trouvé à l'indice $pos"
```

Sortie

```
Maximum 27, trouvé à l'indice 2
```

Question pour la salle : pourquoi l'indice 2 et pas 4 ? Que faudrait-il changer pour obtenir 4 ?

Bash 5 a aussi des **tableaux associatifs** : l'indice est un mot au lieu d'un nombre. Il faut les déclarer avec `declare -A`. Attention, l'ordre de parcours n'est pas garanti.

lundi_assoc.sh

```bash
#!/usr/bin/env bash
declare -A port
port[ssh]=22
port[http]=80
port[https]=443
echo "SSH écoute sur le port ${port[ssh]}"
for service in "${!port[@]}"; do
  echo "$service -> ${port[$service]}"
done
```

Sortie

```
SSH écoute sur le port 22
ssh -> 22
https -> 443
http -> 80
```

Exercices du lundi

Chaque exercice est un script à part. Nommez-les comme dans les exemples. L1 à L3 servent d'échauffement : si le jour 7 de la formation Linux vous a paru facile, commencez à L4.

L1. Pair ou impair NIVEAU 1

Le script reçoit un entier en argument et dit s'il est pair ou impair.

Exemple d'exécution

```
$ ./pair.sh 7
7 est impair
$ ./pair.sh 10
10 est pair
```

L2. Le plus grand des trois NIVEAU 1

Trois entiers en argument, le script affiche le plus grand. Interdit d'utiliser `sort`. Pensez aux nombres négatifs.

Exemple d'exécution

```
$ ./max3.sh 4 19 7
Le plus grand est 19
$ ./max3.sh -3 -8 -1
Le plus grand est -1
```

L3. Table de multiplication NIVEAU 1

Afficher la table de l'entier reçu, de 1 à 10, bien alignée avec `printf`.

Exemple d'exécution

```
$ ./table.sh 7
 7 x  1 =   7
 7 x  2 =  14
 7 x  3 =  21
 7 x  4 =  28
 7 x  5 =  35
 7 x  6 =  42
 7 x  7 =  49
 7 x  8 =  56
 7 x  9 =  63
 7 x 10 =  70
```

L4. Année bissextile NIVEAU 2

Une année est bissextile si elle est divisible par 4 sans l'être par 100, ou si elle est divisible par 400. Testez au moins 1900, 2000, 2024 et 2026.

Exemple d'exécution

```
$ ./bissextile.sh 1900
1900 n'est pas bissextile
$ ./bissextile.sh 2000
2000 est bissextile
$ ./bissextile.sh 2024
2024 est bissextile
$ ./bissextile.sh 2026
2026 n'est pas bissextile
```

L5. Somme et factorielle NIVEAU 2

Pour un entier `n`, calculer la somme 1 + 2 + … + n et la factorielle n! = 1 × 2 × … × n, avec une seule boucle. Puis lancez votre script avec 20, et avec 21. Que se passe-t-il, et pourquoi ?

Exemple d'exécution

```
$ ./somme_fact.sh 5
Somme de 1 à 5 : 15
5! = 120
```

L6. FizzBuzz NIVEAU 2

Afficher les nombres de 1 à `n` (100 par défaut). Les multiples de 3 sont remplacés par Fizz, ceux de 5 par Buzz, ceux de 3 et 5 par FizzBuzz. Astuce : `max=${1:-100}` prend l'argument s'il existe, 100 sinon.

Exemple d'exécution

```
$ ./fizzbuzz.sh 15
1 2 Fizz 4 Buzz Fizz 7 8 Fizz Buzz 11 Fizz 13 14 FizzBuzz
```

L7. Nombre premier NIVEAU 2

Un nombre premier n'a que deux diviseurs : 1 et lui-même. Le script teste l'entier reçu. Il est inutile de chercher des diviseurs au-delà de la racine carrée du nombre : pourquoi ? Écrivez la condition sans calculer de racine.

Exemple d'exécution

```
$ ./premier.sh 1
1 n'est pas premier
$ ./premier.sh 91
91 n'est pas premier (divisible par 7)
$ ./premier.sh 7919
7919 est premier
```

L8. Triangle d'étoiles NIVEAU 2

Afficher un triangle de la hauteur demandée. Bonus : un sapin centré (le nouveau script s'appelle `sapin.sh`).

Exemple d'exécution

```
$ ./triangle.sh 4
*
**
***
****
```

Exemple d'exécution

```
$ ./sapin.sh 4
   *
  ***
 *****
*******
```

L9. Compter les occurrences NIVEAU 1

Le premier argument est la valeur cherchée, les suivants forment la liste. Rangez la liste dans un tableau, puis comptez combien de fois la valeur y apparaît.

Exemple d'exécution

```
$ ./occurrences.sh 3 1 3 5 3 7 3
3 apparaît 3 fois sur 6 valeurs
```

L10. Le deuxième plus grand NIVEAU 3

Trouver la deuxième plus grande valeur **distincte**, en un seul parcours du tableau. Trier est interdit. Prévoir le cas où elle n'existe pas.

Exemple d'exécution

```
$ ./second.sh 14 3 27 8 27 1
Deuxième plus grand : 14
$ ./second.sh 5 5 5
Pas de deuxième plus grand
```

L11. Le juste prix NIVEAU 3

L'ordinateur choisit un nombre entre 1 et 100 (`$(( RANDOM % 100 + 1 ))`) et répond « C'est plus » ou « C'est moins » jusqu'à ce que vous trouviez. Il affiche ensuite le nombre d'essais.

Pour tester sans dépendre du hasard, acceptez un argument facultatif qui fixe le nombre secret : `secret=${1:-$(( RANDOM % 100 + 1 ))}`.

```
$ ./devinette.sh
Votre proposition : 50
C'est moins
Votre proposition : 25
C'est plus
Votre proposition : 37
C'est plus
Votre proposition : 43
Trouvé en 4 essais !
```

Bonus : la partie est perdue au bout de 7 essais. Avec la bonne stratégie, 7 essais suffisent toujours. On verra jeudi pourquoi.

Mémo bash

Besoin	Syntaxe
Créer une variable	`nom="Robin"` (sans espaces)
Lire au clavier	`read -p "Question : " rep`
Arguments du script	`$1`, `$2`, `$#` (nombre), `"$@"` (tous)
Valeur par défaut	`n=${1:-100}`
Calcul	`x=$(( a * 2 + 1 ))`, `(( x++ ))`
Nombre aléatoire de 1 à 100	`$(( RANDOM % 100 + 1 ))`
Décimales	`awk 'BEGIN { printf "%.2f", 7 / 3 }'`
Comparer des nombres	`(( a < b ))`, `(( a == b ))` (plutôt que `-lt`, `-eq`)
Comparer des chaînes	`[[ $s == "oui" ]]`, `[[ -z $s ]]`
Condition	`if ...; then ...; elif ...; then ...; else ...; fi`
Boucle sur une plage	`for i in {1..10}; do ...; done`
Boucle avec variable	`for (( i = 0; i < n; i++ )); do ...; done`
Boucle tant que	`while (( x > 0 )); do ...; done`
Fonction	`f() { local a=$1; echo "$a"; }`
Récupérer le résultat	`r=$(f 42)`
Tableau	`t=(3 1 2)`, `t+=(5)`, `${t[0]}`
Tableau associatif	`declare -A p; p[ssh]=22; ${p[ssh]}`
Lire un fichier	`while IFS=: read -r a b _; do ...; done < fichier`
Taille, éléments, indices	`${#t[@]}`, `"${t[@]}"`, `"${!t[@]}"`
Longueur d'une chaîne	`${#s}`
Sous-chaîne	`${s:debut:longueur}`, `${s: -1}`
Contient	`[[ $s == *"$c"* ]]`
Déboguer	`bash -x script.sh`
Mesurer le temps	`time ./script.sh`